

**A Project Based Learning Report on:
ATM Transaction Integrity Using CRC**

In partial fulfilment of the requirement for Project Based Learning of

BACHELOR OF TECHNOLOGY

In

COMPUTER ENGINEERING

For

Data Communication



**BHARATI VIDYAPEETH (DEEMED TO BE UNIVERSITY) COLLEGE
OF ENGINEERING, PUNE-43**

DEPARTMENT OF COMPUTER ENGINEERING

2026-27

SUBMITTED TO:

Dr. Rohini Khalkar

SUBMITTED BY:

Rishika Kumari (Roll No. 29)

Dibyajyoti Sarkar (Roll No. 35)

Riya Singh (Roll No. 36)

PRN

2514110210

2514110217

2514110218

**BHARATI VIDYAPEETH (DEEMED TO BE UNIVERSITY) COLLEGE
OF ENGINEERING, PUNE-43**

DEPARTMENT OF COMPUTER ENGINEERING



CERTIFICATE

This is to certify that the Project Based Learning report titled "**ATM Transaction Integrity Using CRC**", submitted by **Rishika Kumari, Dibyajyoti Sarkar, Riya Singh** to the Bharati Vidyapeeth (Deemed to be University), College of Engineering, Pune-43 for the award of the degree of **BACHELOR OF TECHNOLOGY** in Computer Engineering is a Bonafide record of the PBL work done by them under my supervision.

Place: Pune

Date: 9/10/26


Dr. Rohini Khalkar


Head
Dept. of Computer Engineering
Bharati Vidyapeeth University
College of Engineering
Pune-Satara Road, Pune-411 043

Acknowledgement

To begin with, we would like to thank our Course Coordinator for providing us this golden opportunity. We are very grateful to our Course Coordinator, **Dr. Rohini Khalkar ma'am**, for her valuable support and guidance.

We are also very grateful to all those who helped us throughout this project. We would like to thank **Dr. Rohini Khalkar**, our Course Coordinator, and our **HOD Dr. S.B. Vanjale** sir for their support and guidance in completing our project-based learning on the topic "**ATM Transaction Integrity Using CRC**". It was a great learning experience.

We would like to take this opportunity to express our gratitude to all the group members, Yash, Tanishka, Anuj, Kushagra and Ishaan. The project-based learning would not have been successful without their cooperation and inputs. Last, but not the least, we are thankful to our parents and families who encourage us to study and who gave us the opportunity to spend our time in learning.

Thanks again to all who helped us.

Index

S.NO	TITLE	PAGE NO.
1	Introduction	1-2
2	Objectives	3
3	System Requirements	4-5
4	Architecture	6-10
5	Implementation	11-14
6	Code	15-19
7	Output	20-24
8	Advantages and Disadvantages	25-26
9	Accuracy and Efficiency	27-28
10	Challenges	29
11	Future Scope and Enhancements	30
12	Conclusion	31
13	References	32

1. Introduction

1.1 Background

Automated Teller Machines (ATMs) are widely used for banking operations such as cash withdrawals and balance inquiries. These transactions involve data transmission between an ATM terminal and a bank server through a communication network. During transmission, noise, interference, or communication errors may alter the data, affecting transaction reliability. Therefore, detecting transmission errors is essential for reliable data communication.

Cyclic Redundancy Check (CRC) is an error-detection technique used to identify accidental changes in transmitted data. It treats binary data as a polynomial and performs modulo-2 division using a predefined generator polynomial. The resulting remainder is appended to the original data to form a transmitted frame. At the receiver, the same division is performed on the received frame. A zero remainder indicates that the frame passes the CRC check, while a non-zero remainder indicates a detected error. However, some error patterns may remain undetected.

The **CRC-Based ATM Transaction Integrity** project demonstrates CRC in a simulated ATM environment, representing the ATM terminal as the sender and the bank server as the receiver. It generates transaction data, calculates CRC bits, simulates transmission, and verifies the received data. The system supports error-free transmission and simulated single-bit, multiple-bit, and burst errors.

Through an interactive web interface, users can observe CRC calculations, verify received frames, and analyse transaction results. The project connects data communication theory with a practical banking scenario and demonstrates the importance of detecting accidental data corruption. It is an educational simulation, not a production banking system, and CRC does not provide encryption or complete transaction security.

1.2 Problem Statement

ATM transactions involve transmitting information between an ATM terminal and a bank server. During transmission, communication errors may alter the data, creating a mismatch between the original and received information. If these errors remain undetected, they may affect the reliability of transaction processing.

This project addresses the need for a reliable method to detect accidental data corruption during ATM transaction transmission. It implements a CRC-based simulation to generate error-detection bits, transmit transaction frames, and verify received data. Different error conditions

are simulated to evaluate CRC's ability to identify corrupted frames and prevent frames that fail verification from being treated as successfully verified within the system.

1.3 Scope of the Project

The **CRC-Based ATM Transaction Integrity** project demonstrates CRC-based error detection through a web-based simulation of communication between an ATM terminal and a bank server.

The scope includes:

1. **ATM Transaction Simulation:** Accepts account ID, transaction type, and transaction amount.
2. **CRC Generation:** Calculates CRC bits using a predefined generator polynomial and appends them to the data frame.
3. **Transmission Simulation:** Demonstrates the transfer of transaction frames from sender to receiver.
4. **Error Injection:** Simulates error-free transmission and single-bit, multiple-bit, and burst errors.
5. **Error Detection and Verification:** Performs CRC verification at the receiving end to determine whether a frame passes the error-detection check.
6. **Transaction History and Results:** Displays verification status and relevant CRC information for analysing transmission outcomes.
7. **Educational Demonstration:** Provides an interactive way to understand CRC calculations, binary transmission, and data integrity.

The project is limited to an educational simulation. It does not perform actual banking operations, transfer money, or communicate with real bank servers. CRC detects accidental transmission errors but does not provide encryption, authentication, or protection against intentional data manipulation.

2. Objectives

The main objective of the **CRC-Based ATM Transaction Integrity** project is to demonstrate how Cyclic Redundancy Check (CRC) can be used to detect errors during the transmission of ATM transaction data between a sender and a receiver.

The specific objectives of the project are:

1. To understand the working principle of Cyclic Redundancy Check (CRC) and its role in data communication.
2. To simulate the transmission of ATM transaction data between an ATM terminal and a bank server.
3. To implement CRC generation using binary data and a predefined generator polynomial.
4. To demonstrate how CRC bits are appended to the original data to form a transmission frame.
5. To simulate different transmission conditions, including error-free transmission, single-bit errors, multiple-bit errors, and burst errors.
6. To verify the received data using CRC and identify frames that fail the error-detection check.
7. To display transaction details, CRC calculations, verification results, and transaction history through an interactive web interface.
8. To analyse the effectiveness of CRC in detecting accidental data corruption and understand its importance in reliable data communication.
9. To provide a practical learning experience by connecting theoretical concepts of computer networks with a real-world ATM transaction scenario.

3. System Requirements

The CRC-Based ATM Transaction Integrity project is a web-based application that simulates ATM transaction data transmission and verifies data integrity using the Cyclic Redundancy Check (CRC) technique. The system requirements are divided into hardware and software requirements.

3.1 Hardware Requirements

The following hardware is sufficient to develop and run the project:

Table 3.1: Hardware Requirements

Component	Minimum Requirement
Processor	Intel Core i3 or equivalent
RAM	4 GB
Storage	500 MB of available space
Display	Standard monitor or laptop display
Input Devices	Keyboard and mouse or touch input
Network	Internet connection for accessing the hosted website

3.2 Software Requirements

Table 3.1: Hardware Requirements

Component	Requirement
Operating System	Windows, Linux, or macOS
Frontend Technologies	HTML, CSS, JavaScript
Runtime Environment	Web browser with JavaScript support
Browser	Google Chrome, Microsoft Edge, Mozilla Firefox, or equivalent
Development Environment	Visual Studio Code or any suitable code editor

Hosting Platform	Render
Application Type	Web-based simulation

3.3 Functional Requirements

The system should provide the following functionalities:

1. Accept ATM transaction details such as account ID, transaction type, and amount.
2. Generate CRC bits using the selected generator polynomial.
3. Construct a data frame for transmission.
4. Simulate different transmission conditions, including error-free and corrupted data.
5. Verify received frames using CRC.
6. Display the verification status and relevant calculation details.
7. Maintain and display transaction history.
8. Perform system accuracy analysis using test cases and display the results supported by the implementation.

3.4 Browser and Platform Compatibility

The application is designed to run through a modern web browser without requiring a separate desktop application installation. It can be accessed from compatible desktop computers, laptops, and mobile devices, subject to browser support and responsive interface design.

These requirements provide the basic environment necessary to develop, execute, and demonstrate the CRC-based ATM transaction simulation.

4. Architecture

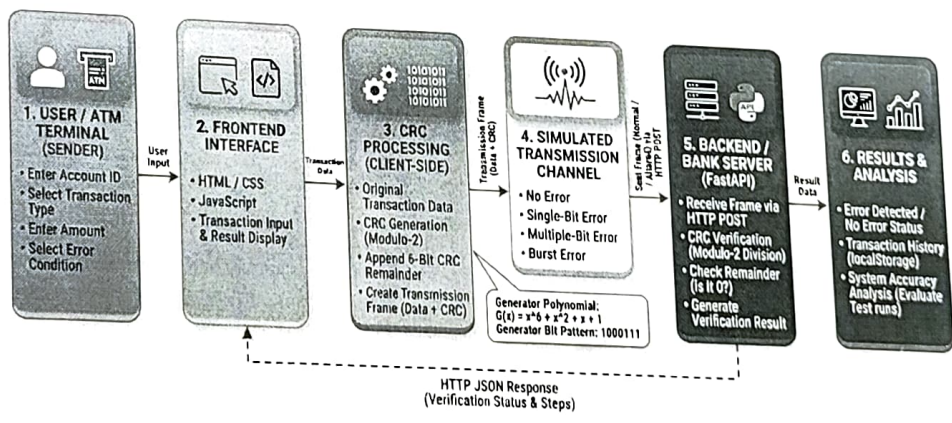
4.1 Overview of the System Architecture

The system architecture of the **ATM Transaction Integrity Using CRC** project describes how transaction data is generated, transmitted, and verified to detect accidental data corruption. It simulates communication between an ATM terminal (sender) and a bank server (receiver) using the Cyclic Redundancy Check (CRC) technique.

The system consists of the user interface, transaction processing module, CRC generation and verification modules, simulated transmission channel, and accuracy analysis module. The frontend accepts transaction details, allows users to select transmission conditions, and displays verification results. The system generates CRC bits, prepares the transmission frame, introduces simulated errors when selected, and verifies the received data.

The application also provides transaction history and accuracy analysis to demonstrate CRC behaviour under different transmission conditions. It is intended for educational purposes and does not perform actual banking operations or transfer real money.

"ATM TRANSACTION INTEGRITY USING CRC"
PBL Project: System Architecture Diagram



4.2 Major Components of the System

The system is divided into the following major components.

4.2.1 Frontend / ATM Terminal

The frontend represents the ATM terminal and acts as the sender. It provides an interface for entering account ID, transaction type, and amount. Users can select transmission conditions, initiate transactions, and view generated frames and verification results.

4.2.2 Transaction Processing Module

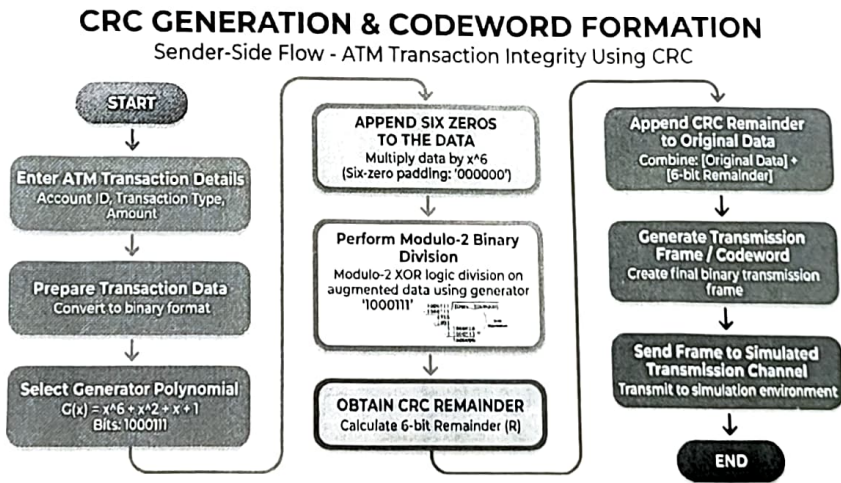
This module collects transaction details and prepares them for CRC processing. It organizes the input data into a suitable representation for CRC generation and maintains the flow of transaction data throughout the simulation.

4.2.3 CRC Generation Module

The CRC generation module calculates error-detection bits using the generator bit pattern 1000111, representing the polynomial:

[
 $G(x)=x^6+x^2+x+1$
]

The transaction data undergoes modulo-2 division, and the resulting remainder is appended to the original data to form the transmission frame, also called the codeword.



4.2.4 Simulated Transmission Channel

The simulated channel represents the communication path between the ATM terminal and the bank server. It allows users to introduce different transmission conditions:

- **No Error:** The original frame remains unchanged.
- **Single-Bit Error:** One bit is modified.

- **Multiple-Bit Error:** Multiple bits are modified.
- **Burst Error:** A group of consecutive bits is affected.

These simulations demonstrate how transmission errors can alter the received frame. The channel operates within the application and does not represent a real banking network.

4.2.5 Backend / Bank Server

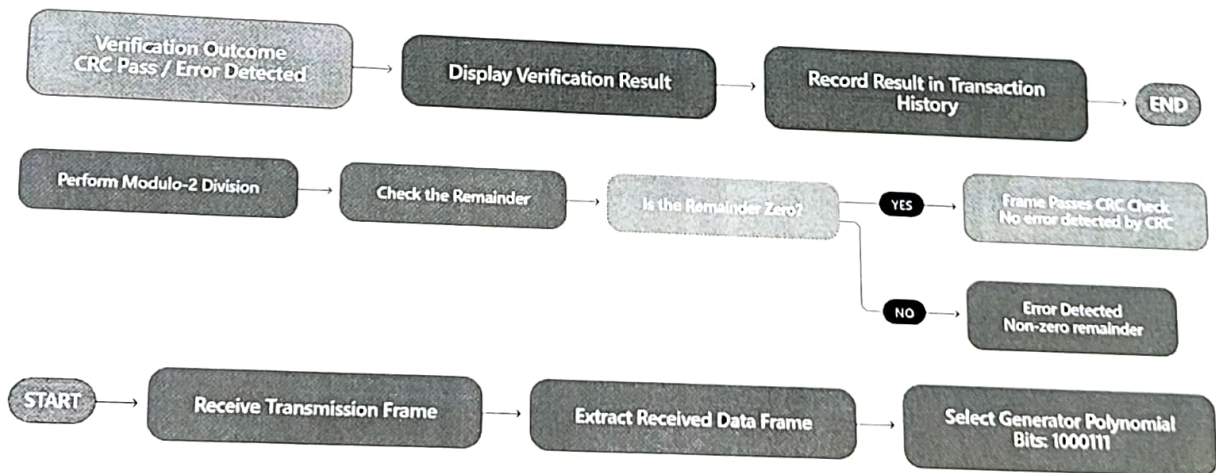
The backend represents the receiving side of the system. It processes the received frame and performs CRC verification. The JavaScript frontend sends transaction data through an HTTP request to the Python FastAPI backend, which returns the verification result for display.

The bank server is simulated and does not connect to an actual bank or authorize real financial transactions.

4.2.6 CRC Verification Module

The verification module performs modulo-2 division on the received frame using the same generator polynomial. A zero remainder indicates that the frame passes the CRC check, while a non-zero remainder indicates a detected error.

However, a zero remainder does not guarantee that the data is error-free, because certain error patterns may remain undetected. CRC provides error detection rather than complete data security.



4.2.7 System Accuracy Analysis Module

This module evaluates CRC behaviour under different transmission conditions, including error-free, single-bit, multiple-bit, and burst errors. It displays information such as the number of test runs, detected errors, undetected errors, and observations.

The error detection rate can be calculated as:

Detection Rate (%)

$$\text{Detection Rate (\%)} = \frac{\text{Corrupted Frames Detected}}{\text{Total Corrupted Frames Tested}} \times 100$$

This detection rate differs from overall accuracy, which may use a different formula. Numerical results should follow the metric actually implemented in the program.

4.2.8 Transaction History and Result Display

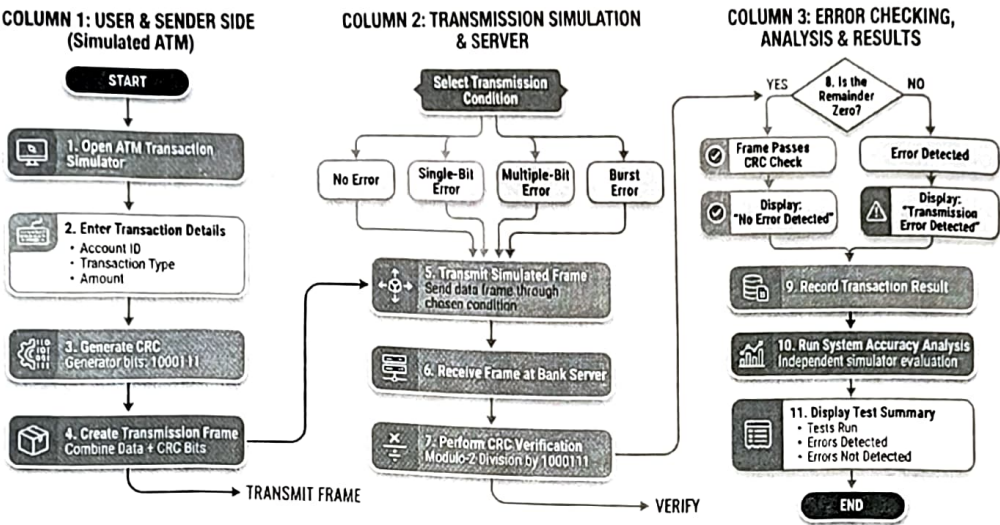
This component displays transaction status, CRC verification outcomes, and relevant frame details. The history feature allows users to review previous simulated transactions. The storage method depends on the implementation; if browser local storage is used, records remain in the user's browser rather than a central database.

4.3 Data Flow of the System

The system follows a sequential process from transaction entry to final verification:

1. The user enters transaction details through the ATM interface.
2. The transaction processing module prepares the data.
3. The CRC module calculates the remainder using the generator polynomial.
4. The remainder is appended to form the transmission frame.
5. The simulated channel introduces the selected error condition, if any.
6. The received frame is sent to the backend for verification.
7. The receiver performs modulo-2 division and checks the remainder.
8. The system displays whether the frame passes the CRC check or an error is detected.
9. The result is recorded in transaction history, where supported.
10. The accuracy analysis module summarizes results from multiple test cases.

“COMPLETE SYSTEM EXECUTION FLOWCHART: ATM TRANSACTION INTEGRITY USING CRC”
PBL Project: Web-Based ATM Transaction Integrity Simulator Execution Flow



4.4 Working Principle of the Architecture

The architecture separates transaction input, CRC generation, transmission simulation, and receiver-side verification into distinct stages. This modular structure makes the error-detection process easier to understand.

During CRC generation, the original data is divided by the generator polynomial using modulo-2 arithmetic. The remainder is appended to form the transmission frame. The simulated channel may then modify the frame according to the selected error condition.

At the receiving end, the frame is checked using the same generator polynomial. A zero remainder indicates that the frame passes the CRC check, whereas a non-zero remainder indicates a detected error. The result is displayed and may be included in the transaction history and accuracy analysis.

4.5 Architectural Limitations

The project demonstrates CRC error detection in a controlled simulation. It does not implement complete banking infrastructure, real financial transaction processing, or communication with actual bank servers.

CRC detects certain types of transmission errors but does not provide encryption or authentication. Therefore, it cannot independently protect transactions against fraud, unauthorized access, or deliberate data manipulation. The accuracy results also depend on the test cases and conditions used and do not guarantee performance in real banking networks.

5. Implementation

5.1 Introduction

The implementation of the ATM Transaction Integrity Using CRC project simulates the transmission of ATM transaction data from an ATM terminal to a bank server. It allows users to create transactions, simulate transmission errors, verify received data, and examine the results using Cyclic Redundancy Check (CRC).

The web-based application demonstrates how CRC can detect accidental data corruption during transmission. It is intended for educational purposes and does not perform actual banking operations or transfer real money.

5.2 User Interface Implementation

The application provides a web-based interface for operating and monitoring the simulation. Its major components are:

- **ATM Terminal:** Collects the account ID, transaction type, and transaction amount.
- **Transmission Configuration:** Allows the selection of no-error, single-bit, multiple-bit, or burst-error conditions.
- **Bank Server (Receiver):** Verifies the received codeword using CRC.
- **CRC Analysis:** Displays error-detection results and observations.
- **Transaction History:** Allows users to review simulated transaction results.
- **System Accuracy Analysis:** Evaluates error-detection performance across multiple test cases.

The interface combines these components into a single environment, allowing users to observe the transaction process from data entry to receiver-side verification.

5.3 Transaction Data Formation

The process begins when the user enters the required transaction details. The application combines the account ID, transaction type, and amount into a transaction data string and converts it into binary form.

The system then calculates the CRC remainder using the selected generator polynomial. The resulting CRC bits are appended to the original binary data to form the transmitted codeword.

This codeword contains the transaction data and additional bits used by the receiver to check for transmission errors.

5.4 CRC Generation and Transmission

The project uses the CRC generator bit pattern **1000111**, corresponding to the polynomial:

$$G(x) = x^6 + x^2 + x + 1$$

CRC generation uses modulo-2 division, in which XOR operations are performed on the binary data. The remainder obtained from this division forms the CRC value.

The CRC bits are appended to the original data to produce the codeword. The application then simulates transmission to the bank receiver, where the received codeword is checked using the same generator polynomial.

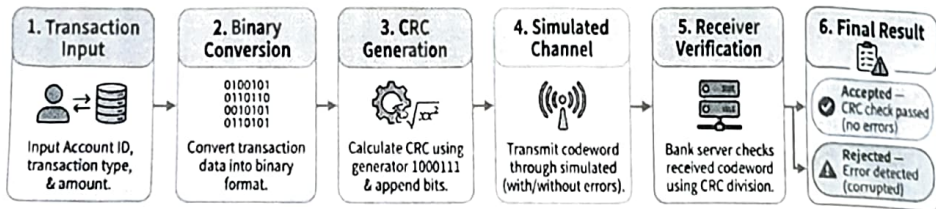


Figure 5.1 – CRC-Based Transaction Processing Flow

5.5 Simulation of Transmission Errors

The application supports four transmission conditions:

1. **No Error:** The codeword is transmitted without modification.
2. **Single-Bit Error:** One bit of the codeword is changed.
3. **Multiple-Bit Error:** More than one bit is changed.
4. **Burst Error:** A group of bits within a portion of the codeword is affected.

These conditions demonstrate how different forms of data corruption can influence CRC verification. All errors are generated within the software simulation and do not represent actual faults in a physical banking network.

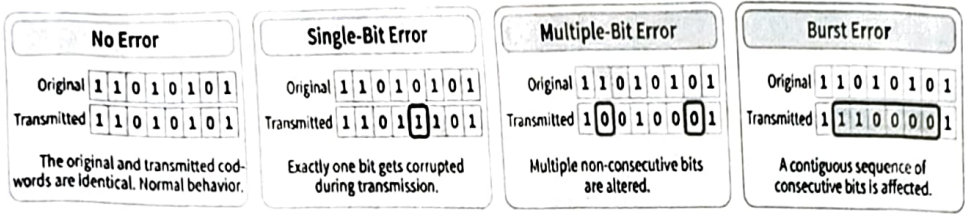


Figure 5.2 – Simulated Transmission Error Types

5.6 Receiver-Side CRC Verification

After transmission, the receiver performs modulo-2 division on the received codeword using the generator polynomial.

- **Zero remainder:** The codeword passes the CRC check, and no error is detected.
- **Non-zero remainder:** The system detects an error and marks the codeword as rejected by the simulator.

A zero remainder does not guarantee that the data is completely error-free because certain error patterns can remain undetected. CRC is an error-detection mechanism, not an error-correction or data-encryption method.

5.7 Transaction History and Result Display

The application displays the verification outcome and provides transaction history for reviewing previous simulations. Users can compare the selected transmission condition with the receiver's result.

The CRC analysis section records information such as the number of test runs, detected errors, undetected errors, and observations for different transmission conditions. This helps users examine the behaviour of CRC during repeated tests.

5.8 System Accuracy Testing

The application includes a batch-testing feature that evaluates multiple codewords under different transmission conditions. The results indicate whether the corrupted codewords are detected by the CRC verification process.

The error-detection rate is calculated using the following formula:

$$\text{Detection Rate (\%)} = \frac{\text{Number of Corrupted Frames Detected}}{\text{Total Number of Corrupted Frames Tested}} \times 100$$

Only corrupted frames are included in the denominator. Error-free frames should be evaluated separately to determine whether they are correctly accepted.

The measured result depends on the generated test cases. This test evaluates the simulator's CRC behaviour and does not establish the security or reliability of a real ATM network.

5.9 Summary

The implementation combines transaction data formation, CRC generation, transmission error simulation, receiver-side verification, transaction history, and batch accuracy testing in a single web-based application. It demonstrates the basic principles of CRC-based error detection through an interactive ATM transaction scenario.

6. Code

This section describes the core implementation of the ATM Transaction Integrity Using CRC project. It covers the HTML interface, CSS styling, JavaScript transaction processing, CRC generation, transmission error simulation, and Python FastAPI backend. The system verifies received codewords using modulo-2 division and supports batch verification for accuracy testing.

6.1 HTML: ATM Transaction Interface

The HTML file (index.html) defines the structure of the web application. It contains the ATM transaction form, simulated transmission channel, bank receiver panel, transaction history, and CRC analysis section.

The transaction form collects the account ID, transaction type (withdrawal or deposit), and transaction amount. A transaction-generation button initiates processing. The remaining interface displays the generated CRC codeword, simulated transmission conditions, verification results, and analysis data.

6.2 CSS: Styling and Layout

The stylesheet (style.css) controls the appearance and layout of the application. It defines the colour scheme, panels, input fields, buttons, status indicators, transmission animation, and responsive layout.

Blue represents the ATM and data-related elements, amber highlights the transmission process, and green or red indicates verification outcomes. The styling provides a consistent interface and helps users distinguish between different stages of the transaction.

6.3 JavaScript: CRC Generation

The JavaScript file (script.js) generates the CRC codeword using the generator pattern 1000111, corresponding to the polynomial (x^6+x^2+x+1). Six zeros are appended to the original binary data before modulo-2 division.

```
const GENERATOR = "1000111";
const CRC_BITS = GENERATOR.length - 1;

function generateCRC(dataBits) {
    const augmented = dataBits + "0".repeat(CRC_BITS);
    const detail = crcDivisionDetailed(augmented);
```

```

    return {
        remainder: detail.remainder,
        codeword: dataBits + detail.remainder
    };
}

```

The function calculates the CRC remainder and appends it to the original data to form the transmitted codeword.

6.4 JavaScript: Transaction Processing

The `generateTransaction()` function validates the user's input, combines the transaction details, and converts them into binary before CRC generation.

```

const dataText = accId + "|" + txnType + "|" + amount;
const binary = textToBinary(dataText);
const crc = generateCRC(binary);

```

```

currentTxn = {
    accId: accId,
    txnType: txnType,
    amount: amount,
    binary: binary,
    remainder: crc.remainder,
    codeword: crc.codeword
};

```

This code stores the transaction details and generated codeword for the transmission process. Input validation is performed before these statements.

6.5 JavaScript: Transmission Error Simulation

The simulator supports no-error, single-bit, multiple-bit, and burst-error conditions. The `introduceError()` function modifies the codeword according to the selected condition.

```

const result = introduceError(
    currentTxn.codeword,
    typeKey,
    config
);

```

```

const transmitted = result.transmitted;
const positions = result.positions;

```

The modified codeword is sent to the backend for CRC verification. The positions identify the bits altered during the simulated transmission.

6.6 Python: FastAPI Bank Server

The backend is implemented in app.py using FastAPI. It receives transaction information and the transmitted codeword.

```
GENERATOR = "1000111"
CRC_BITS = len(GENERATOR) - 1

class VerifyRequest(BaseModel):
    transaction_id: str
    transaction_type: str
    codeword: str
    generator: str
```

The request model defines the information expected by the verification endpoint.

6.7 Python: Backend CRC Verification

The `crc_divide()` function performs modulo-2 division using XOR operations. It returns the calculated remainder and the selected division steps.

```
remainder, steps = crc_divide(codeword, GENERATOR)
no_error = remainder == "0" * CRC_BITS
```

A zero remainder means that the codeword passes the CRC check, while a non-zero remainder indicates a detected error. However, some error patterns may remain undetected.

The `/verify` endpoint validates the binary codeword and returns the verification result in JSON format.

6.8 Frontend–Backend Communication

The frontend sends the transmitted codeword to the FastAPI backend using an HTTP POST request.

```
const response = await fetch("/verify", {
  method: "POST",
  headers: {
    "Content-Type": "application/json"
  },
  body: JSON.stringify({
    transaction_id: label,
    transaction_type: currentTxn.txnType,
```

```
        codeword: transmitted,
        generator: GENERATOR
    })
});
```

The frontend processes the server's response and displays the CRC remainder, verification status, and transaction result.

6.9 Python: Batch Verification

The `/verify_batch` endpoint processes multiple codewords in one request for accuracy testing.

```
results = []

for item in req.items:
    codeword = item.codeword.strip()
    remainder, _ = crc_divide(codeword, GENERATOR)

    results.append({
        "id": item.id,
        "detected": remainder != "0" * CRC_BITS
    })

return {"results": results}
```

The endpoint returns a detection result for each codeword. The frontend uses these results to analyse the simulator's performance under different transmission conditions.

6.10 Deployment Configuration

The `requirements.txt` file specifies the Python dependencies required by the application, including FastAPI and Uvicorn.

The `render.yaml` file defines the deployment configuration, including the Python runtime, dependency installation command, server startup command, and health-check path. Uvicorn runs the FastAPI application on the port provided by the deployment environment.

6.11 Summary

The project combines HTML and CSS for the user interface, JavaScript for transaction processing and transmission error simulation, and Python FastAPI for backend verification. The `/verify` endpoint handles individual transactions, while `/verify_batch` supports multiple test

ATM Transaction Integrity Using CRC

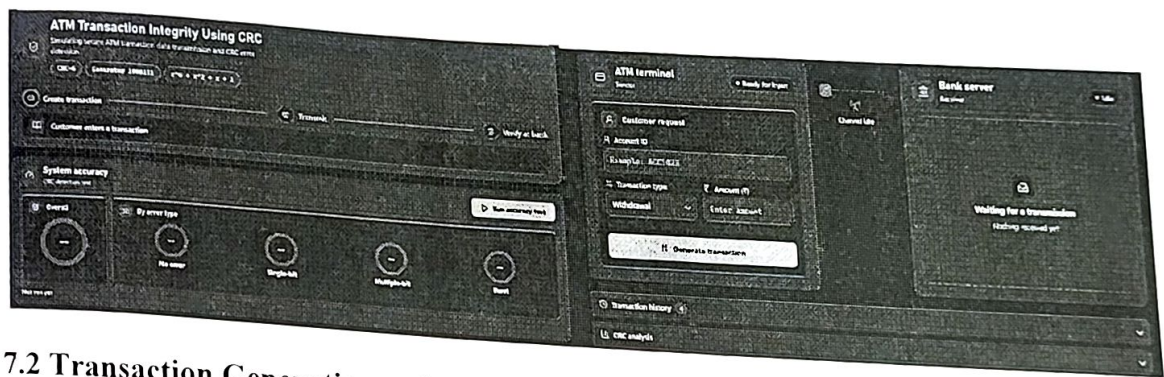
cases. Together, these components demonstrate the use of CRC for detecting accidental corruption in simulated ATM transaction data.

7. Output

7.1 Home Page / ATM Transaction Interface

The home page provides an interface for entering transaction details, including account ID, transaction type, and transaction amount. The user can generate a transaction and proceed with the CRC-based transmission simulation.

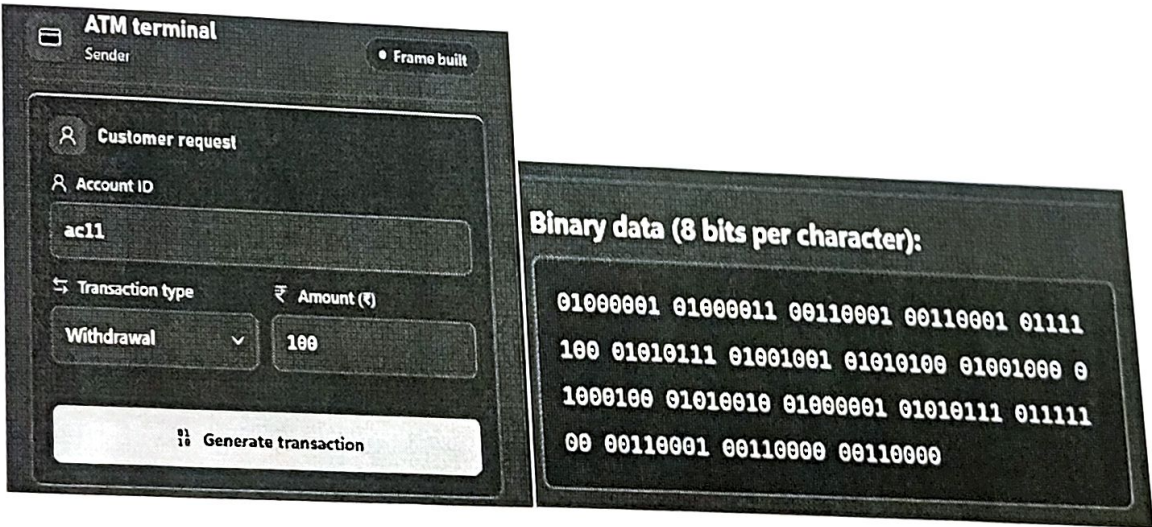
Output:

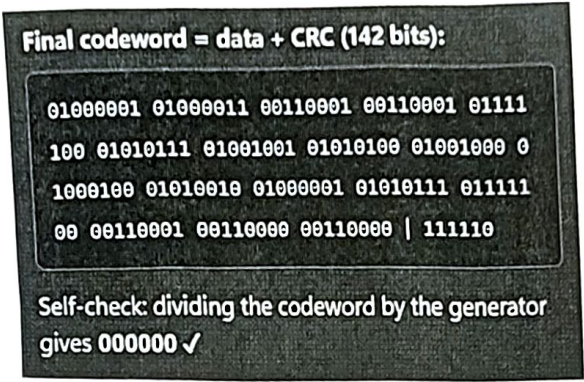


7.2 Transaction Generation and CRC Codeword

After entering the transaction details, the system generates the corresponding binary data and calculates the CRC bits using the selected generator polynomial, $G(x)=x^6+x^2+x+1$, represented by the bit pattern 1000111. The CRC remainder is appended to the original data to form the transmitted codeword.

Output:



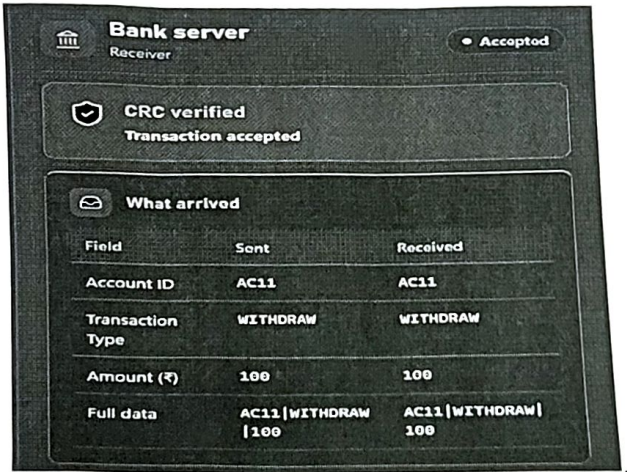


7.3 Error-Free Transmission

In this test, the transaction is transmitted without introducing any errors. The receiving bank performs modulo-2 division on the received codeword using the generator polynomial. If the remainder is zero, the transaction passes the CRC verification.

Expected result: No error detected; the transaction is accepted by the verification system.

Output:

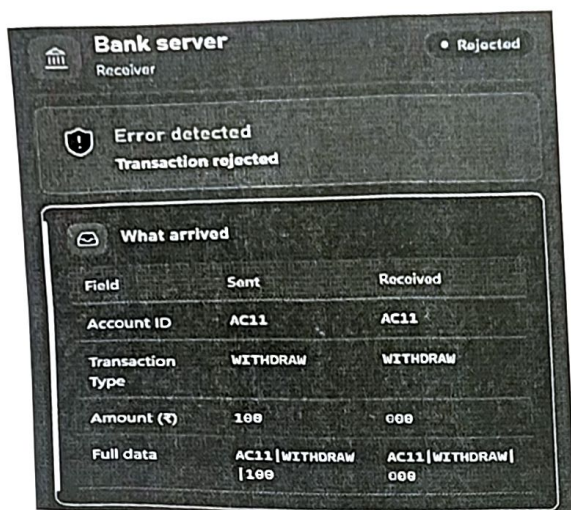


7.4 Single-Bit Error Detection

In this test, a single bit of the transmitted codeword is altered to simulate a transmission error. The receiving bank verifies the modified codeword using CRC.

Expected result: Error detected; the transaction is rejected by the verification system.

Output:

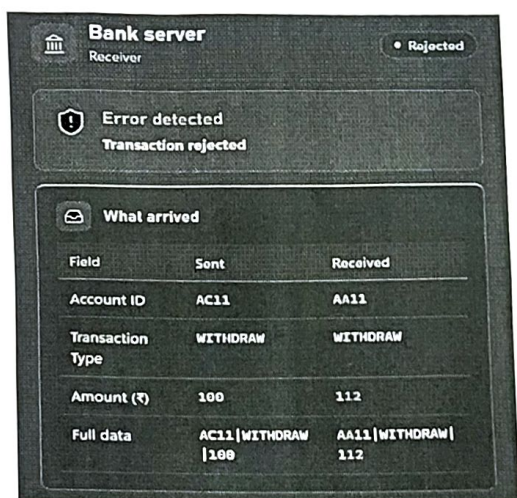


7.5 Multiple-Bit Error Detection

In this test, multiple bits of the transmitted codeword are altered during transmission. The receiver performs CRC verification to determine whether the received codeword contains a detectable error.

Expected result: The system detects the error when the modified codeword produces a non-zero CRC remainder.

Output:



7.6 Burst Error Detection

In this test, a group of consecutive bits is modified to simulate a burst error during data transmission. The receiver checks the received codeword using the CRC generator polynomial.

Expected result: The system detects the burst error if the received codeword produces a non-zero remainder.

Amount (₹)	100	100
Full data	AC11 WITHDRAW 100	AC11 WITHDRAW 100

Transaction History and Analysis

The system maintains a history of simulated transactions and displays their transmission outcomes. The analysis table records the number of runs, detected errors, and observations for different transmission conditions.

ut:

Transaction history				
Transaction	Type	Error	CRC result	Status
TXN001	Withdrawal	No Error	Valid	Accepted
TXN002	Deposit	Burst Error	Error	Rejected
TXN003	Withdrawal	Single-Bit Error	Error	Rejected
TXN004	Withdrawal	Burst Error	Valid (error missed)	Accepted
TXN005	Withdrawal	No Error	Valid	Accepted
TXN006	Withdrawal	Single-Bit Error	Error	Rejected
TXN007	Withdrawal	Multiple-Bit Error	Error	Rejected
TXN008	Withdrawal	Burst Error	Error	Rejected

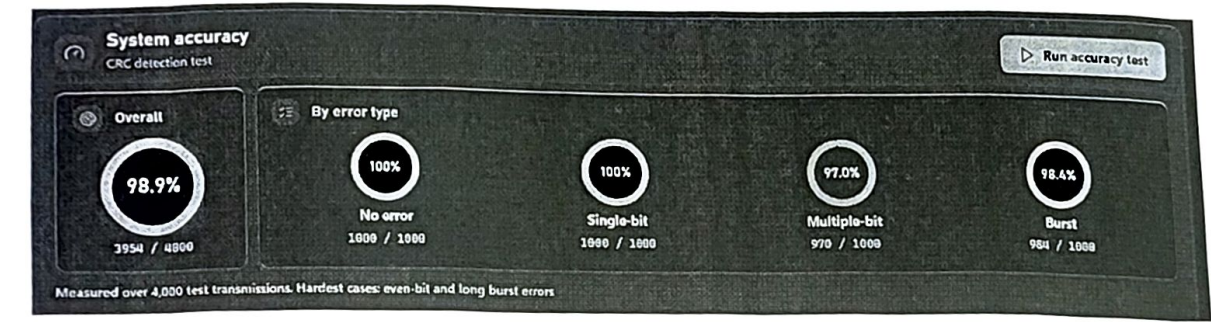
CRC analysis				
Condition	Runs	Error detected	Not detected	Observation
No Error	2	0	2	No false alarms ✓
Single-Bit Error	2	2	0	All errors detected
Multiple-Bit Error	1	1	0	
Burst Error				

ATM Transaction Integrity Using CRC

The accuracy testing feature performs multiple test cases for different transmission conditions. The results are processed by the backend verification system and displayed in the analysis panel.

The results help evaluate how effectively the CRC mechanism detects simulated transmission errors.

Output:



7.9 Result Summary

The project demonstrates the use of Cyclic Redundancy Check (CRC) for detecting accidental data corruption during simulated ATM transaction transmission. The system generates CRC codewords, introduces configurable transmission errors, verifies received data through a backend server, and displays the verification results.

The simulation illustrates the role of CRC in checking data integrity. CRC verification does not provide encryption or guarantee the authenticity of a transaction.

8. Advantages and Disadvantages

8.1 Advantages

1. **Error Detection:** CRC helps detect accidental changes in transmitted data by checking the remainder obtained through modulo-2 division.
2. **Reliable Data Verification:** The receiving system verifies the received codeword before accepting the simulated transaction.
3. **Detection of Different Error Patterns:** The application supports simulations involving single-bit, multiple-bit, and burst errors to demonstrate CRC-based error detection.
4. **Fast Verification:** CRC verification uses binary operations such as XOR, making the checking process computationally efficient.
5. **Automated Testing:** The system provides batch accuracy testing to evaluate CRC verification across multiple test cases.
6. **User-Friendly Interface:** The web-based interface allows users to enter transaction details, simulate transmission errors, and view verification results.
7. **Frontend and Backend Integration:** The project combines a JavaScript-based interface with a Python FastAPI backend to process and verify received codewords.
8. **Transaction History and Analysis:** The application records simulated transmission results and displays analysis for different error conditions.
9. **Educational Value:** The project provides a practical demonstration of CRC generation, codeword formation, transmission error simulation, and receiver-side verification.

2 Disadvantages

1. **No Error Correction:** CRC detects many types of transmission errors but does not identify and correct the corrupted bits.
2. **Possibility of Undetected Errors:** Certain error patterns may produce a zero remainder and remain undetected by CRC.
3. **No Data Encryption:** CRC does not encrypt transaction information or prevent unauthorized users from reading it.
4. **No Authentication:** CRC alone cannot verify the identity of the sender or confirm that a transaction request is authorized.
5. **Dependence on the Generator Polynomial:** Error-detection capability depends on the selected generator polynomial and the characteristics of the transmitted data.

6. **Simulation-Based Implementation:** The project simulates ATM transaction transmission and verification; it is not connected to an actual ATM network or banking system.
7. **Limited Security Protection:** CRC is designed for accidental error detection, not for protecting against deliberate data manipulation by an attacker.
8. **Network and Server Dependency:** The deployed web application requires a working connection to the backend server for transaction verification and batch testing.

8.3 Summary

The project demonstrates that CRC is an efficient method for detecting accidental data corruption during simulated ATM transaction transmission. Its main advantages are fast verification, support for different error simulations, and automated result analysis. However, CRC cannot correct errors or provide encryption and transaction authentication. Therefore, a real banking system would require additional security mechanisms and transaction-validation procedures.

9. Accuracy and Efficiency

9.1 Accuracy Analysis

The accuracy of the ATM Transaction Integrity Using CRC system is evaluated by testing different transmission conditions, including error-free transmission, single-bit errors, multiple-bit errors, and burst errors. The application performs repeated test cases and verifies the received codewords using the CRC generator polynomial 1000111.

During verification, the backend performs modulo-2 division on the received codeword. A zero remainder indicates that the codeword passes CRC verification, whereas a non-zero remainder indicates a detected error.

The application records the number of test runs, detected errors, undetected errors, and observations for each transmission condition. These results help evaluate the effectiveness of CRC in identifying corrupted data.

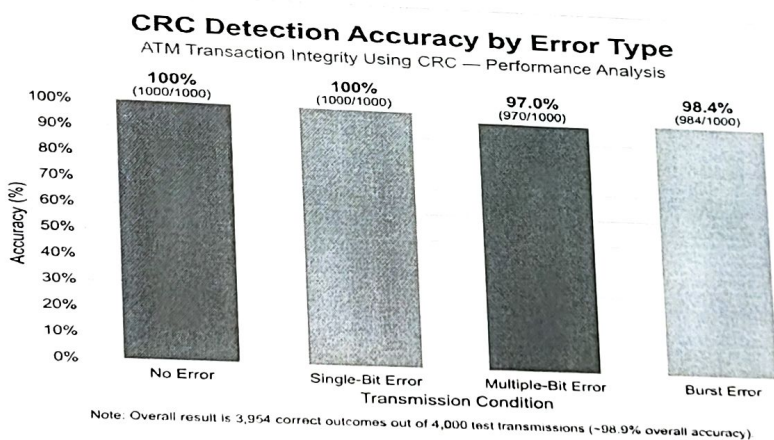


Figure 9.1: CRC Detection Accuracy by Error Type.

The detection rate can be calculated using the following formula:

Error Detection Rate (%)

$$\text{Error Detection Rate} = \frac{\text{Number of Corrupted Frames Detected}}{\text{Total Number of Corrupted Frames Tested}} \times 100$$

Error-free frames should not be included in the denominator when calculating the error detection rate. They can instead be evaluated separately to determine whether valid frames are correctly accepted.

9.2 Efficiency Analysis

The efficiency of the system is evaluated based on the time and computational effort required to generate CRC codewords, simulate transmission errors, and verify received data.

1. **CRC Generation:** The system generates CRC bits using binary operations and modulo-2 division.
2. **Verification Speed:** The backend checks the received codeword by performing modulo-2 division using the configured generator polynomial.
3. **Automated Batch Testing:** The application supports batch verification of multiple codewords, reducing the need to test each codeword individually.
4. **Resource Utilization:** The project uses a lightweight Python FastAPI backend and a JavaScript-based frontend, making it suitable for an educational simulation.
5. **Result Processing:** The application collects verification results and displays transaction history and error-analysis statistics.

The actual processing time depends on the codeword length, number of test cases, network latency, and backend response time. Therefore, numerical performance claims should be made only after measuring the running application.

9.3 Interpretation of Results

The accuracy and efficiency analysis demonstrates how CRC can be used to verify data integrity during simulated transaction transmission. A higher error detection rate indicates that more of the tested corrupted frames were identified by the system.

However, a frame that passes CRC verification is not guaranteed to be completely error-free, because some error patterns may produce a zero remainder. Similarly, CRC verification does not provide encryption or authentication.

9.4 Conclusion

The project provides a practical method for evaluating CRC-based error detection through configurable transmission errors and automated batch testing. Its accuracy can be assessed using the detected and undetected error counts, while its efficiency can be evaluated through verification time and processing performance. The recorded results help demonstrate the usefulness and limitations of CRC in maintaining data integrity.

10. Challenges

During the development of the ATM Transaction Integrity Using CRC project, several challenges were encountered in implementing CRC verification, simulating transmission errors, and integrating the frontend with the backend.

10.1 CRC Algorithm Implementation

Implementing modulo-2 division using XOR operations required careful handling of binary strings and the generator polynomial to ensure correct CRC generation and verification.

10.2 Transmission Error Simulation

Simulating single-bit, multiple-bit, and burst errors required separate logic to modify transmitted codewords while maintaining valid binary data.

10.3 Frontend and Backend Integration

Connecting the JavaScript frontend with the Python FastAPI backend required proper handling of HTTP requests and responses to display verification results correctly.

10.4 Error Detection and Result Interpretation

Interpreting CRC results required distinguishing between zero and non-zero remainders. However, certain error patterns may remain undetected even when the remainder is zero.

10.5 Accuracy Testing and Analysis

Evaluating different transmission conditions required automated testing. Batch verification helped process multiple test cases and analyse the results.

10.6 Deployment and Accessibility

Deploying the application required configuring the Python environment, installing dependencies, and ensuring proper communication between the frontend and the hosted backend.

10.7 User Interface and Result Presentation

Displaying transaction details, verification status, history, and accuracy statistics clearly required careful interface design and organization.

10.8 Conclusion

Addressing these challenges provided practical experience in CRC implementation, error simulation, frontend-backend integration, automated testing, and web application deployment.

11. Future Scope and Enhancements

The project can be enhanced to improve its security, performance, and practical applicability through the following developments.

11.1 Real-Time Communication: Simulate real-time data transmission between an ATM terminal and a bank server to demonstrate transaction verification in a network environment.

11.2 Enhanced Security: Integrate encryption, digital signatures, and authentication alongside CRC to protect sensitive data and prevent unauthorized modifications.

11.3 Multiple CRC Standards: Support CRC-8, CRC-16, and CRC-32 to compare their error-detection capabilities under different transmission conditions.

11.4 Advanced Error Simulation: Introduce configurable error probabilities, random error patterns, and different burst lengths for comprehensive testing.

11.5 Performance Analysis: Measure execution time, processing speed, and memory usage, and present the results using graphs and statistical summaries.

11.6 Improved Interface and Reporting: Add downloadable reports, exportable test results, interactive graphs, and detailed verification logs.

11.7 Database Integration: Store transaction simulations, verification outcomes, and historical test results for future analysis.

11.8 Scalability and Deployment: Improve support for multiple users and larger test batches through automated testing, monitoring, structured logging, and robust error handling.

evaluate the CRC. The project combines HTML, CSS, and JavaScript for the user interface with Python and Flask for backend processing. Its web-based implementation makes the CRC verification process interactive and helps users understand the practical working of error detection in data communication systems.

Through this project, the importance of data integrity checking and the role of CRC in identifying accidental transmission errors are demonstrated. The project also highlights the limitations of CRC, as certain error patterns may remain undetected and CRC does not provide encryption or authentication.

Overall, this project provides a practical learning experience in error-detection algorithms, front-end-backend integration, automated testing, and web application deployment. With future enhancements such as support for additional CRC standards, advanced error simulation, persistent data storage, and stronger security mechanisms, the system can be developed into a comprehensive data-integrity testing platform.

13. References

- Tanenbaum, A. S., and Wetherall, D. J., *Computer Networks*, 5th Edition, Pearson. Reference for data communication, error detection, and CRC concepts.
- Python Software Foundation, "Python Documentation." Available at: <https://docs.python.org/3/>
- FastAPI, "FastAPI Documentation." Available at: <https://fastapi.tiangolo.com/>
- Mozilla Developer Network (MDN), "HTML: HyperText Markup Language." Available at: <https://developer.mozilla.org/en-US/docs/Web/HTML>
- Mozilla Developer Network (MDN), "CSS: Cascading Style Sheets." Available at: <https://developer.mozilla.org/en-US/docs/Web/CSS>
- Mozilla Developer Network (MDN), "JavaScript Documentation." Available at: <https://developer.mozilla.org/en-US/docs/Web/JavaScript>
- Render, "Deploy a FastAPI App." Available at: <https://render.com/docs/deploy-fastapi>
- Render, "Web Services." Available at: <https://render.com/docs/web-services>
- Project Website, "ATM Transaction Integrity Using CRC." Available at: <https://crc-mechanism-for-atm-transaction.onrender.com/>
- Project Source Code, *ATM Transaction Integrity Using CRC* — HTML, CSS, JavaScript, Python FastAPI backend, and deployment configuration developed as part of this project.